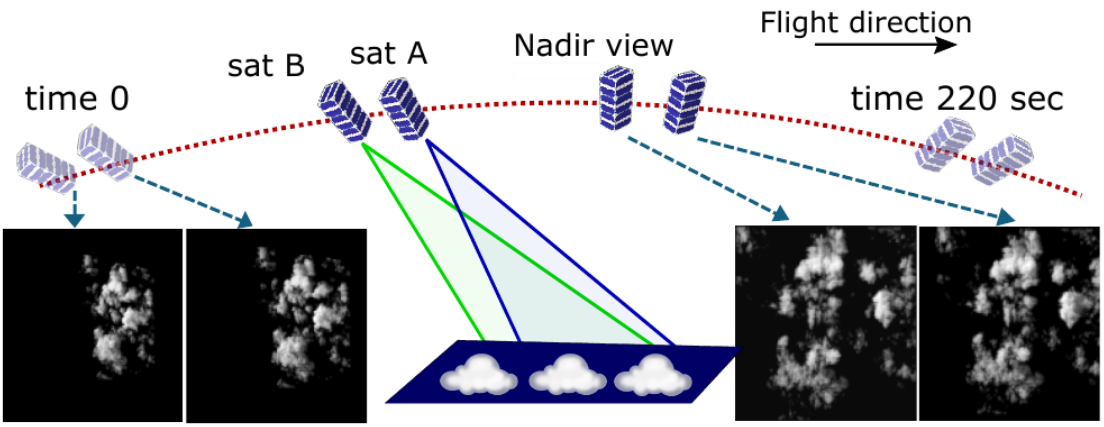
Inferring Vertical Air Flow by Spaceborne Imaging

A Walkthrough of the Code



Release repository `Inferring-Vertical-Air-Flow` (WACV 2027)

Dor Danino, Udi Gal, Vadim Holodovsky, Daniel Rosenfeld,
Barry Lynn, Céline Cornet, Yoav Y. Schechner

This document follows a single V_z map from the LES cloud field, through rendering, dataset construction, the network, the loss and training, to evaluation and the real-data experiment. Each section points to the file and function that implements that step.

November 2026

Contents

1 The problem and the big picture **2**

2 Data generation **3**

2.1 From microphysics to extinction: `calc_beta.py`, `les_to_volumes.py` 3

2.2 Rendering: `mitsuba_renderer.py`, `render_dataset.py` 3

2.3 Ground truth: `velocity_maps.py` 4

2.4 Sequences, noise and the spatial split: `build_kfold_dataset.py` 4

3 Loading data: `vertical_airflow/dataset.py` **5**

4 The model: `vertical_airflow/models.py` **5**

4.1 Forward pass, step by step 5

4.2 The ConvLSTM is causal 6

5 The loss: `vertical_airflow/losses.py` **7**

5.1 The spatial weight M 7

5.2 Bin loss (Eq. 6) 7

5.3 Power loss (Eq. 7) and the schedule 8

6 Configuration: `configs/` and `utils.py` **8**

7 Training: `train.py` **8**

7.1 Two stages 8

7.2 Checkpoint selection 9

7.3 Resuming and monitoring 9

8 Evaluation: `evaluate.py` **9**

8.1 The K-fold ensemble 9

8.2 Which pixels are counted 9

9 Real data: SEAC4RS, `real_data/seac4rs/` **10**

1 The problem and the big picture

The goal is to infer the *vertical air velocity* V_z (m/s) inside and around shallow cumulus clouds from what a pair of satellites can see: a short sequence of stereo images taken during one orbital overpass. Two kinds of targets are learned, each by its own model:

- **Cloud envelope**: V_z at the first cloudy voxel hit by each camera ray (the visible cloud surface). It is defined only on cloudy pixels.
- **Horizontal layers**: V_z on a horizontal plane at a fixed altitude Z (500, 900, 1000 or 1500 m), including the clear air around the clouds.

No real V_z ground truth exists at this scale, so the models are trained on physics-based synthetic data. A Large-Eddy Simulation (LES) supplies a 4D cloud field. A Monte Carlo path tracer (Mitsuba 3) renders it from two satellites moving along a realistic orbit, and the LES velocity field supplies the labels. Figure 1 is the paper's overview, and the code follows it closely.

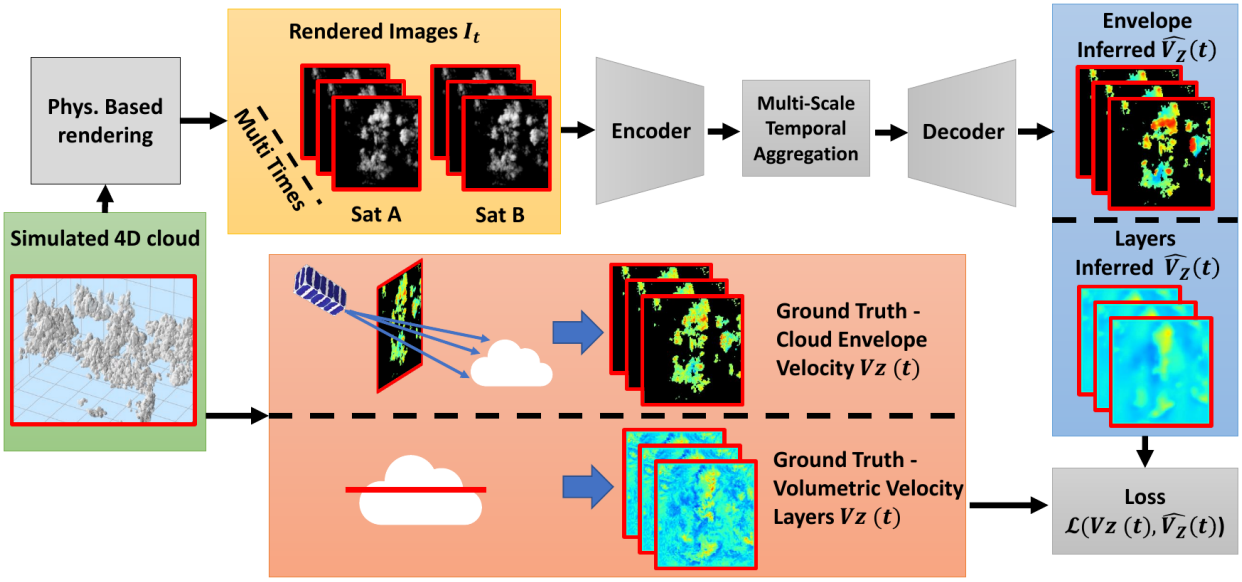
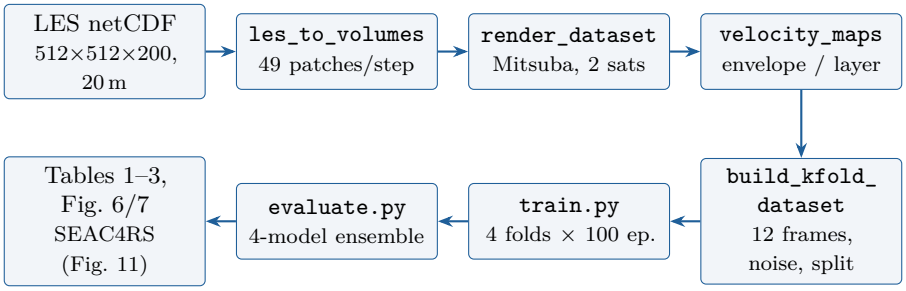


Figure 1: Paper Fig. 2. A simulated 4D cloud is rendered into stereo sequences (top), and ground truth is extracted at the cloud envelope or on horizontal layers (bottom). An encoder, a multi-scale temporal aggregator and a decoder map the images to $\widehat{V_z}(t)$.

Repository map

Path	Role
`data_generation/`	LES → volumes → Mitsuba renders → V_z maps → K-fold `.npz` files
`vertical_airflow/dataset.py`	`NPZSequenceDataset`: loading, normalisation, cloud mask
`vertical_airflow/models.py`	MiT-B1 + ConvLSTM U-Net
`vertical_airflow/losses.py`	Bin loss (Eq. 6), power loss (Eq. 7), L1, and the loss schedule
`vertical_airflow/utils.py`	Config inheritance, model factory, checkpoint loading
`vertical_airflow/ensemble.py`	K-fold ensemble used by the figure scripts
`train.py`	Two-stage K-fold training of one experiment
`evaluate.py`	Ensemble evaluation: Tables 1–3 and the Fig. 6 scatter
`configs/`	One YAML file per paper experiment, all inheriting `base.yaml`
`figures/`	Fig. 6 and Fig. 7
`real_data/seac4rs/`	AirMSPI + DC-8 real-data experiment (Sec. 5, Fig. 11)
`scripts/`	Batch training/evaluation, long-run queue, status, paper comparison

End-to-end flow



2 Data generation

All of this lives in `data_generation/`. It runs once and is expensive: rendering the full dataset took about 3 days on 10 RTX 3090 GPUs. Users who download the released `.npz` files can skip this section.

2.1 From microphysics to extinction: `calc_beta.py`, `les_to_volumes.py`

The renderer needs the extinction coefficient β_{ext} in every voxel. `process_cloud_vars` computes it from the LES microphysics (paper Eqs. 1–5): air density from the ideal gas law, liquid water content, mean droplet volume, effective radius, and finally $\beta_{\text{ext}} = 3\,\text{LWC}/(2\rho_w R_{\text{eff}})$ (Mie extinction with $Q_{\text{ext}} = 2$):

```
rho_air = (P * M_AIR) / (K_B * T)                          # Eq. 1
LWC = QN * rho_air                                         # Eq. 2
droplet_vol = (LWC / (NC + 1e-15)) / RHO_WATER             # Eq. 3
reff_m = ((3 * droplet_vol) / (4 * np.pi)) ** (1 / 3)      # Eq. 4
beta_ext = (3 * LWC) / (2 * RHO_WATER * reff_m + 1e-15)    # Eq. 5
```

`les_to_volumes.py` cuts each 512×512 LES time step into a **7×7 grid of overlapping 128×128×200 patches** (stride 64). Each patch is pickled with `U, V, W, beta_ext`, and its index is `NNN = row*7 + col`. This grid index is reused later for the spatial train/val/test split.

2.2 Rendering: `mitsuba_renderer.py`, `render_dataset.py`

`MitsubaRenderer` builds a Mitsuba 3 scene with:

- a heterogeneous medium with extinction β_{ext}, single-scattering albedo 1 and a Henyey–Greenstein phase function ($g = 0.85$);
- volumetric path tracing (`volpath`, unlimited depth, 8192 spp);
- the sun as a directional emitter, and a dark diffuse "ocean" (albedo 0.03);
- perspective cameras at the satellite positions read from `assets/overpass_2satellites.csv`.

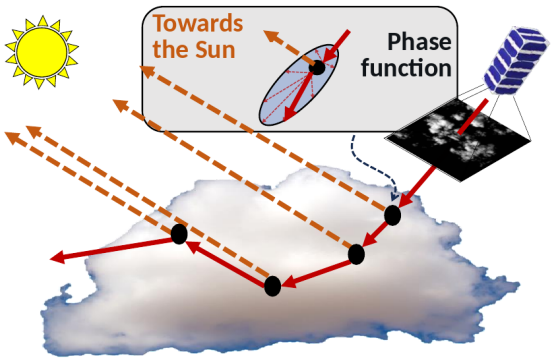


Figure 2: Paper Fig. 4: Monte Carlo path tracing within a cloud.

The overpass CSV has 12 acquisition times (0, 20, …, 220 s) × 2 satellites. Each row gives sun and satellite angles and the satellite position in ENU kilometres. The key design choice in `render_dataset.py` is how time is mapped:

Time mapping

Sorted LES time folders are assigned *cyclically* to the 12 acquisition times: folder k is imaged at time index k mod 12. Twelve consecutive LES steps (20 s apart) therefore form one overpass, in which the cloud evolution, the satellite motion and the sun direction all advance together. With `--start/-end` the work can be split across machines, and the time index is always computed from the position in the *full* folder list, so the split doesn't change it.

2.3 Ground truth: `velocity_maps.py`

Two modes produce the label maps at the same 256×256 resolution as the renders:

- `--mode envelope`: for every pixel of satellite view k, march the camera ray through the volume in 20 m steps and take V_z of the **first voxel with** $\beta_{\text{ext}} > 0$. Clear-sky pixels are NaN (later set to 0).
- `--mode layer -height Z`: sample V_z on the horizontal plane at altitude Z, seen from nadir. The result includes clear air.

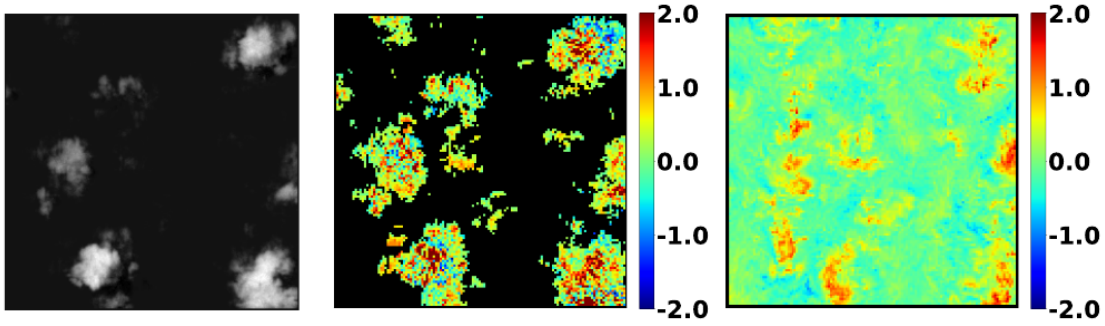


Figure 3: Paper Fig. 3. Left: a rendered image. Middle: envelope V_z, defined only on the cloud. Right: a horizontal-layer V_z, dense over the whole field (m/s).

2.4 Sequences, noise and the spatial split: `build_kfold_dataset.py`

This script turns the renders and one set of label maps into training files:

1. **Sequences.** Twelve consecutive time folders form one sequence ($T = 12$, 220 s).
2. **Resize.** Images and maps are resized from 256 to 128 px with area interpolation.
3. **Camera noise** (`sensor_noise`): radiance → electrons, plus dark current and Gaussian read noise ($\sigma^2 = 5.29\,e^2$), clipping to a 10 600 e^- full well and 10-bit quantisation. No photon noise is added, because the Monte Carlo rendering noise is already at least as large (see the paper's supplementary material).
4. **Spatial split** of the 7×7 patch grid, so no test patch overlaps a training patch:

```
VAL_BLOCKS  = [((0, 1), (0, 2)), ((0, 1), (4, 6)), ((2, 3), (0, 6)), ((5, 6), (0, 2))]
TEST_CELLS  = {(5, 4), (5, 5), (5, 6), (6, 4), (6, 5), (6, 6)}        # fixed
IGNORE_CELLS = {(4, c) for c in range(3, 7)} | {(5, 3), (6, 3)}      # halo around test
```

Each fold takes one validation block and drops that block's 8-connected halo, because neighbouring patches overlap by 64 px. All remaining cells are training data. The output for each target (`envelope`, `layer_500m`, ...) looks like this:

```
data/layer_1000m/
  test_w.npz                                  # shared test set (414 sequences)
  fold_01_val_r0-1_c0-2/{train_w,val_w}.npz   # 4 folds, named after the val block
  ...
# X: float32 [N, 12, 2, 128, 128]   radiance of satellites A and B
# Y: float32 [N, 12, 1, 128, 128]   V_z in m/s
```

3 Loading data: `vertical_airflow/dataset.py`

`NPZSequenceDataset` is small, but three of its details affect every number reported later.

```
CLOUD_THRESHOLD = 1.1   # raw radiance above which a pixel is "cloudy"
TARGET_NORM = 0.95      # max |V_z| of the training set is mapped to 0.95

self.norm_const = max(np.max(self.X), 1.0)              # input scale
max_abs = max(np.max(self.Y), np.abs(np.min(self.Y)))
self.scale = max_abs / TARGET_NORM                       # symmetric label scale

def __getitem__(self, idx):
    x = torch.from_numpy(self.X[idx])
    mask = (x[:, 0:1] > CLOUD_THRESHOLD).float()         # cloud mask from satellite A
    x = x / self.norm_const
    y = torch.from_numpy(np.clip(self.Y[idx] / self.scale, -1.0, 1.0).astype(np.float32))
    return x, y, mask
```

- **The cloud mask M is computed from the image**, not from the labels. It comes from satellite A's raw radiance before normalisation. The same mask is used by the loss and by the evaluation, so "masked" metrics mean cloudy as seen by the camera.
- **Labels are scaled symmetrically** so that the largest $|V_z|$ in the training set becomes 0.95. The model predicts in this normalised space, and `denormalize` multiplies by `scale` to return to m/s.
- **Validation and test sets must reuse the training statistics**: `val_ds.use_normalization_of(train_ds)`. Each fold has its own training set and therefore its own `scale`. This matters for the ensemble (Section 8).
- **Monocular mode** (`use_one_satellite: true`) keeps only channel 0 (satellite A), so $C = 1$.

4 The model: `vertical_airflow/models.py`

The main model is the class `PretrainedTemporalUNetMitB1`. It is built from parts of the `segmentation-models-pytorch` library, with ConvLSTMs inserted between the encoder and the decoder. The input is $[B, T, C, H, W]$ and the output is $[B, T, 1, H, W]$, one V_z map per frame.

4.1 Forward pass, step by step

```
def forward(self, x_seq):
    B, T, C, H, W = x_seq.shape
    x_flat = x_seq.view(B * T, C, H, W)          # (1) fold time into the batch
    x_flat = self.input_adapter(x_flat)          # (2) 1x1 conv: C (1 or 2) -> 3 channels
    features = self.encoder(x_flat)              # (3) MiT-B1, frame by frame

    # (4) bottleneck: project -> ConvLSTM over time -> expand
    bottleneck = self.bottleneck_proj(features[-1])
```

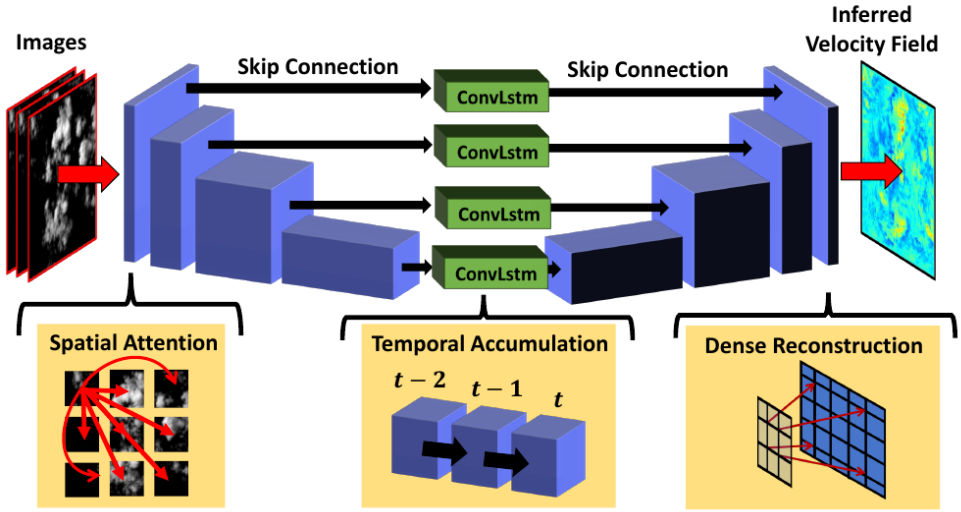


Figure 4: Paper Fig. 5. A Mix Vision Transformer encoder (spatial attention), ConvLSTM blocks on the bottleneck and on every skip connection (temporal accumulation), and a convolutional decoder (dense reconstruction).

```
    seq = bottleneck.view(B, T, -1, h, w)
    out, _ = self.lstm([seq[:, t] for t in range(T)])
    features[-1] = self.dropout(self.bottleneck_expand(stack(out).view(B * T, ...)))

    # (5) the same for every skip connection, each with its own ConvLSTM
    for i, use_lstm in enumerate(self._lstm_skip_map): ...

    decoder_out = self.decoder(*features)        # (6) MAnet decoder, frame by frame
    output = self.head(self.dropout(decoder_out))
    return output.view(B, T, 1, H, W), None
```

1. **Time folded into the batch.** The encoder and decoder are purely spatial, so all $B \cdot T$ frames go through them in one call.
2. **Input adapter.** MiT-B1 is pretrained on 3-channel ImageNet images. A learned 1×1 convolution maps the stereo pair (or the single mono channel) to 3 channels, so the pretrained weights stay usable.
3. **Encoder.** MiT-B1 (the SegFormer backbone) returns a pyramid of feature maps at strides 1–32.
4. **Temporal bottleneck.** The deepest features (512 channels) are projected by a 1×1 conv to 64 channels, unfolded back to $[B, T, \dots]$, and passed through a `ConvLSTM` along time. The result is projected back to 512 channels.
5. **Temporal skips.** Every non-empty skip level gets its own project $\rightarrow$ ConvLSTM $\rightarrow$ expand block. This is the "multi-scale temporal aggregation" of Fig. 1. The MiT-B1 encoder's levels have (3, 0, 64, 128, 320, 512) channels: level 0 is the adapted input itself and gets a ConvLSTM too, and the empty stride-2 level is skipped through `_lstm_skip_map`.
6. **Decoder and head.** The `MAnet` decoder with channels (512, 256, 128, 64, 32) and a segmentation head with 1 output channel. Dropout2d ($p = 0.3$) is applied to the temporally processed features and before the head.

4.2 The ConvLSTM is causal

```python
class ConvLSTMCell(nn.Module):
    def forward(self, x, state=None):
        gates = self.conv(torch.cat([x, h], dim=1))     # one 3x3 conv for all 4 gates
        i, f, g, o = torch.chunk(gates, 4, dim=1)
        c_next = sigmoid(f) * c + sigmoid(i) * tanh(g)
        h_next = sigmoid(o) * tanh(c_next)
```

`ConvLSTM.forward` loops $t = 0 \dots T-1$ in order and carries (h, c) forward, so the output at frame t depends only on frames $\leq t$. The SEAC4RS script relies on this to run the model online (Section 9).

Checkpoint compatibility

Module and parameter names are kept exactly as in the original training code, so the released checkpoints load with `strict=True`. The unused `RefinerHead` exists for the same reason; it is always disabled by `build_model`. Renaming anything in `models.py` breaks the released weights.

5 The loss: `vertical_airflow/losses.py`

V_z is heavily concentrated around 0 m/s, while the strong updrafts and downdrafts are the physically interesting part. A plain L1 loss learns to predict near zero everywhere. The loss design addresses this imbalance.

5.1 The spatial weight M

`spatial_weights` turns the cloud mask into per-pixel weights, according to `loss.use_mask`:

`use_mask`	Used by	Weight M
`true`	envelope	1 on cloudy pixels, 0 elsewhere
`slice_mask`	layers	cloud mask of the middle frame ($t = 5$), dilated 5×5, applied to all frames; 1 inside, `unmasked_weight`= 0.9 outside
`false`	–	1 everywhere

5.2 Bin loss (Eq. 6)

Paper, Eq. 6

$$\mathcal{L}_{\text{bin}} = \frac{\left\| \mathbf{M} \odot \mathbf{\Omega}_{V_z} \odot (\widehat{\mathbf{V}}_z - \mathbf{V}_z) \right\|_1}{\left\| \mathbf{M} \odot \mathbf{\Omega}_{V_z} \right\|_1}$$

$\mathbf{\Omega}_{V_z}$ is a per-pixel weight inversely proportional to how many pixels in the batch fall into the same 0.1 m/s velocity bin.

`bin_loss` recomputes the histogram *for every batch*:

```python
y_ms = dataset.denormalize(y).clamp(bin_min, bin_max - 1e-6)      # bins are in m/s
y_flat = y_ms[hist_sel]                                            # cloudy / mask region only
bin_counts[i] = ((y_flat >= lo) & (y_flat < lo + BIN_WIDTH)).sum()
bin_weights[non_empty] = y_flat.numel() / bin_counts[non_empty]   # inverse frequency
bin_weights[non_empty] /= bin_weights[non_empty].mean()           # mean 1
bin_weights = torch.clamp(bin_weights, max=MAX_BIN_WEIGHT)        # cap at 100
...
return (abs_diff * pixel_w * m).sum() / (pixel_w * m).sum()
```

The bin range `[bin_min, bin_max]` is the min/max of the fold's training labels, fixed in `train.py`. The histogram is taken over the pixels that matter (cloudy, or the dilated mask region), so clear air

doesn't dominate the counts.

5.3 Power loss (Eq. 7) and the schedule

$$\mathcal{L}_{\text{power}} = \left\| (1 + 4\,|\mathbf{V}_z|^8) \odot (\widehat{\mathbf{V}}_z - \mathbf{V}_z) \right\|_1$$

is computed on *normalised* velocities, where $|V_z| \leq 0.95$. It barely changes the weight of typical pixels and strongly up-weights the extremes. It is smooth and stable early in training, so it serves as a warm-up:

```
class LossSchedule:          # schedule = "power_to_bins" (default)
    def mode(self, epoch):
        if epoch <= 10:  return "power", 0.0
        if epoch <= 15:  return "interp", (epoch - 10) / 5     # (1-a)*L_power + a*L_bin
        return "bins", 1.0
```

6 Configuration: `configs/` and `utils.py`

Every experiment is a few lines that inherit `base.yaml`. `load_config` merges `base:` recursively, so a config only states what differs:

```
# configs/envelope.yaml
base: base.yaml
name: envelope
data: {kfold_dir: envelope,
       target: envelope}
loss: {use_mask: true}
```

```
# configs/layer_500m_mono.yaml
base: base.yaml
name: layer_500m_mono
data: {kfold_dir: layer_500m,
       use_one_satellite: true}
```

Paper result	Configs
Table 1, Fig. 6 (left), Fig. 7	`envelope`, `layer_500m`, `layer_1000m`, `layer_1500m`
Table 3 (monocular)	`envelope_mono`, `layer_{500,1000,1500}m_mono`
Sec. 5, Fig. 11 (SEAC4RS)	`layer_900m_mono`

`build_model(backbone, in_channels, use_conv_lstm)` is the single model factory. The architecture of a checkpoint is stored inside the checkpoint itself (`ckpt["config"]`), so `load_model_from_checkpoint` doesn't need the YAML file.

7 Training: `train.py`

```
python train.py --config configs/layer_1000m.yaml            # all 4 folds
python train.py --config configs/envelope.yaml --skip-existing --status-file STATUS.txt
python train.py --config configs/envelope.yaml --epochs 1 1  # smoke test
```

For every `fold_*` directory, `train_fold` trains one model on `train_w.npz`, selects it on `val_w.npz` and reports single-model metrics on the shared `test_w.npz`.

7.1 Two stages

	Epochs	Encoder	Batch	AdamW lr (ReduceLROnPlateau patience)
Stage 1	1–20	frozen	32	10^{-3} (3)
Stage 2	21–100	trainable	16	$3 \cdot 10^{-4}$ (5)

Stage 1 trains only the adapter, ConvLSTMs and decoder on top of frozen ImageNet features. At the switch, the best Stage-1 weights (if any) are loaded, the encoder is unfrozen, new data loaders and a new optimizer are created, and the "best" tracker is reset. Mixed precision (`autocast` + `GradScaler`) and gradient clipping at 1.0 are used throughout.

7.2 Checkpoint selection

```
if loss_fn.is_final_loss(epoch) and va[0] < best_val:      # only once the bin loss is active
    torch.save({"model_state": ..., "config": {"type", "in_channels", "use_one_satellite",
                "use_conv_lstm", "stage"}, "train_config": cfg, "val_loss": ...}, out_path)
```

A checkpoint is saved only while the *final* loss is active (epoch ≥ 16 for `power_to_bins`), so validation losses of different loss functions are never compared. Because the tracker is reset at Stage 2, the saved file is the best *Stage-2* model.

7.3 Resuming and monitoring

After each fold, the per-fold test metrics are written to `<out>/<name>/train_metrics.json`. `--skip-existing` skips a fold only if it appears in that file *and* its `.pt` exists. A fold that was interrupted halfway is therefore retrained from scratch. `--status-file` rewrites a one-line progress summary with an ETA after every epoch. `scripts/retrain.sh` queues experiments in tmux, and `scripts/status.sh` prints the queue, the current epoch, finished folds, GPU and disk usage.

8 Evaluation: `evaluate.py`

```
python evaluate.py --config configs/layer_1000m.yaml --checkpoints checkpoints/layer_1000m --
    scatter
#   unmasked  MAE=0.19  ME=+0.03  STD=0.38 ...
#   masked    MAE=0.37  ME=+0.07  STD=0.64 ...
```

8.1 The K-fold ensemble

The reported number is not a single model. The 4 fold models each predict, each prediction is de-normalised with *that fold's* training scale, and the predictions are averaged in m/s:

```
fold_preds = [ds.denormalize(m(x)[0].cpu().numpy()) for m, ds in zip(models, norms)]
pred = np.mean(np.stack(fold_preds), axis=0)
```

`vertical_airflow/ensemble.py` wraps the same logic in an `Ensemble` class, which also returns the per-pixel standard deviation across folds. The figure scripts use it.

8.2 Which pixels are counted

`eval_masks` defines the statistics:

- **Envelope:** cloudy pixels only (`masked`).
- **Layers:** `unmasked` = all pixels, and `masked` = the dilated middle-frame cloud mask, the same region the loss emphasises. Table 1 reports both as "unmasked / masked".

MAE, ME (mean error, the bias), STD of the error and RMSE are written to `eval_metrics.json`. `--gain-mismatch 0.03` multiplies camera B by 1.03 to test robustness to radiometric calibration

errors. `--test-npz` evaluates on another sequence, such as the cross-domain test.

Statistic [m/s]	Envelope	$Z = 500\,\mathrm{m}$	$Z = 1000\,\mathrm{m}$	$Z = 1500\,\mathrm{m}$
MAE	0.74	0.22 / 0.34	0.19 / 0.37	0.16 / 0.27
Mean error	0.21	−0.07 / −0.03	0.02 / 0.06	0.04 / 0.07
STD	1.02	0.31 / 0.44	0.38 / 0.64	0.32 / 0.56

Paper Table 1: test-set statistics of the 4-model ensemble (layers: unmasked / masked).

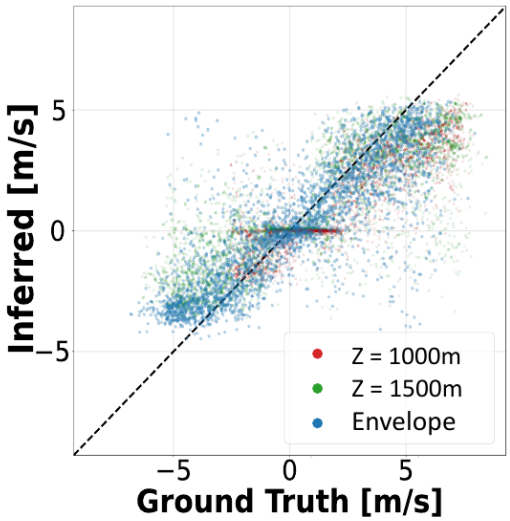


Figure 5: Paper Fig. 6 (left). Inferred vs. ground-truth V_z, produced by `figures/fig6_scatter.py` from the `--scatter` points. `balanced_scatter` samples up to 5 points per 0.02 m/s ground-truth bin, so rare high velocities are visible.

9 Real data: SEAC4RS, `real_data/seac4rs/`

On 12 August 2013, the ER-2 aircraft carried AirMSPI, a multi-angle imager, over a cloud field while the DC-8 aircraft measured vertical wind in situ. AirMSPI has a single camera that views the scene from changing angles. This resembles one satellite in a monocular setting, so the `layer_900m_mono` model is used.

1. **Inputs** (ready to use, in `real_data/seac4rs/data/`): `airmspi.npz` holds the 9 AirMSPI views (355 nm intensity, cropped around a fixed geographic centre, resized to 128 px and normalised jointly to $[0, 1]$) as X of shape $[1, T, 2, 128, 128]$, with the single camera duplicated into both channels. `dc8.csv` holds the DC-8 MMS 1 Hz vertical velocity in the AirMSPI time window.
2. `infer_airmspi.py`: **causal inference**. For each t the ensemble sees frames $1..t$ and keeps the prediction for frame t, as an online system would. Background pixels are removed, and all cloudy-pixel predictions are pooled.
3. `compare_distributions.py`: there is no pixel-to-point correspondence between the images and the aircraft, so the *distributions* are compared: density overlay, Q-Q plot, Wasserstein distance (0.146 m/s) and Q-Q MAE (0.143 m/s).

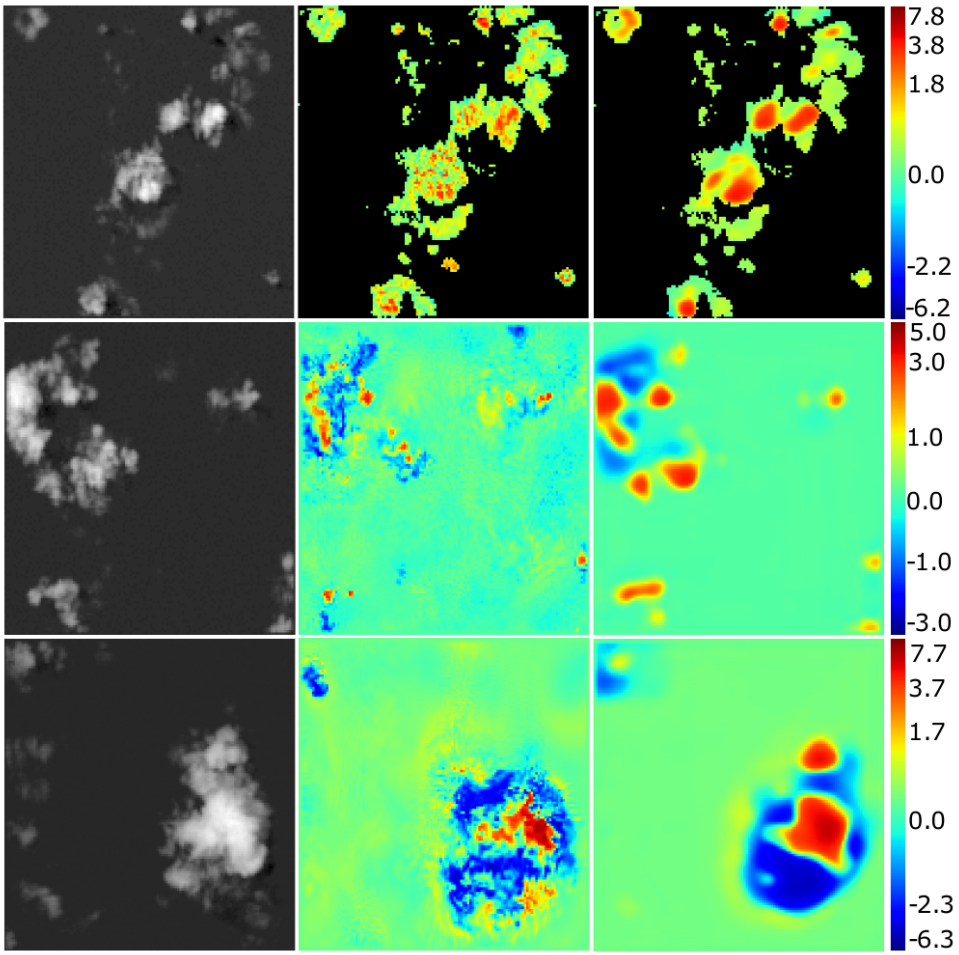


Figure 6: Paper Fig. 7, produced by `figures/fig7_qualitative.py`. Rows: cloud envelope, $Z = 1000\,\mathrm{m}$, $Z = 1500\,\mathrm{m}$. Columns: input image, ground truth, inferred (m/s).

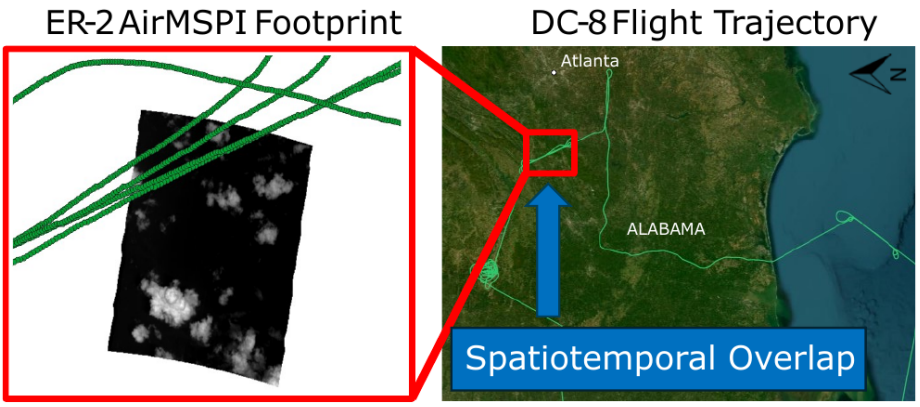


Figure 7: Paper Fig. 10: the coordinated SEAC4RS flights.

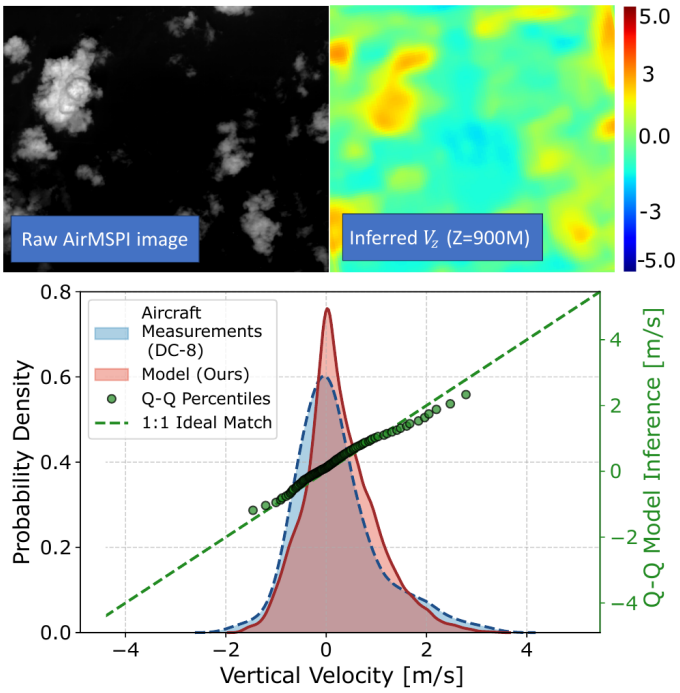


Figure 8: Paper Fig. 11: the inferred V_z distribution vs. DC-8 in-situ measurements.